

HelixPlay

HelixPlay

Transformez n'importe quelle machine GPU en une solution de cloud gaming personnelle.

Résumé

HelixPlay est une plateforme de cloud gaming auto-hébergeable qui transforme toute machine équipée de GPU en un hôte de streaming distant, offrant une expérience de jeu de niveau console sur les clients desktop, mobiles, TV et navigateurs. Conçue comme un monorepo centré sur Go et composé de 46 sous-modules, elle est personnalisable en marque blanche pour les partenaires.

Description courte

HelixPlay est une plateforme de cloud gaming auto-hébergeable, ouverte et personnalisable en marque blanche. Elle transforme toute machine équipée de GPU en un hôte de streaming distant et diffuse une expérience de jeu de niveau console sur les clients desktop, mobiles, TV et navigateurs via WebRTC/QUIC, avec un cœur Go et une pile client Wails/Flutter/Angular.

Description longue

HelixPlay est une plateforme de cloud gaming conçue comme un monorepo centré sur Go et composé de 46 sous-modules Git. Elle prend n'importe quel PC gaming que vous possédez déjà et le transforme en un hôte de streaming, offrant une expérience de niveau console sur les clients desktop, mobiles, TV et navigateurs — auto-hébergeable, ouverte et personnalisable en marque blanche pour les partenaires. Le message est clair : votre matériel, votre service, votre marque, sans cloud tiers intermédiaire.

Son choix de conception déterminant repose sur la convergence des clients en une triple pile architecturale — un pari technique qui porte ses fruits à tous les niveaux. Une application desktop Wails, une application mobile/TV Flutter et un client web Angular s'appuient sur un *unique* cœur Go, compilé en WASM pour le navigateur, de sorte que le comportement est écrit une seule fois et partagé sur toutes les plateformes, plutôt que dupliqué en trois versions. En dessous se trouve le chemin média en temps réel : Capture → Encodage → Paquetisation → Transmission → Décodage → Rendu, connecté à des solutions de capture natives (DXGI / ScreenCaptureKit / PipeWire) et à des encodeurs matériels (NVENC / QSV / AMF / VideoToolbox), permettant à la GPU de gérer la charge lourde. Le transport s'effectue via WebRTC (Pion v4), QUIC (quic-go) et des datagrammes personnalisés UDP, choisis pour leur latence plutôt que pour leur simplicité. Le cœur backend gère les sessions, les locataires, le catalogue et l'authentification ; un agent hôte prend en charge la capture, l'encodage et le transport en périphérie ; et mDNS/rendezvous assure la découverte pour que les clients trouvent leur hôte sans configuration manuelle.

HelixPlay est architecturée dès le départ pour le SaaS en marque blanche — thématization par locataire, filtrage du catalogue, OAuth2 et facturation — permettant à un partenaire de déployer un service entièrement personnalisé, bien au-delà d'un simple rebranding. Et elle est native aux conteneurs dans les moindres détails : chaque service, base de données, build, test et analyse s'exécute dans des conteneurs, ce qui rend l'ensemble de la plateforme reproductible à déployer et à vérifier. Comme le reste de la famille Helix, elle repose sur une constitution anti-bidon où un test vert doit garantir un comportement réel et utilisable par l'utilisateur final — et non un simple mock validé.

Pourquoi nous l'avons créée

Le cloud gaming commercial est fermé, centralisé et loué. HelixPlay a été conçue pour que quiconque possédant une machine GPU puisse faire tourner son propre hôte de streaming — ouvert, auto-hébergé et personnalisable en marque blanche — plutôt que de dépendre d'un service tiers.

Contenu

Pourquoi c'est un tournant

Il fusionne trois éléments que les services commerciaux maintiennent séparés : l'auto-hébergement sur du matériel que vous contrôlez, un cœur Go partagé alimentant trois piles clientes pour que les fonctionnalités soient déployées partout

simultanément, et une multi-location en marque blanche.
Résultat : un partenaire peut lancer un service de cloud gaming entièrement personnalisé sur *ses propres* GPU — maîtrisant l'expérience, les utilisateurs et l'économie — au lieu de revendre de la capacité sur le cloud d'un tiers et de se heurter à ses limites.

Ce qui est innovant

- **Convergence des trois piles clientes** — Wails, Flutter et Angular reposant sur un seul cœur Go (WASM dans le navigateur), de sorte que les versions desktop, mobile, TV et web partagent une implémentation unique au lieu de trois versions divergentes.
- **SaaS auto-hébergeable et en marque blanche** — personnalisation par locataire, filtrage du catalogue, OAuth2 et facturation intégrés, ce qui permet à la plateforme d'être livrée comme un produit personnalisable, et non comme une simple démonstration.
- **Transport moderne à faible latence** — WebRTC (Pion), QUIC et un UDP sur mesure, couplés à une sélection d'encodeurs matériels par plateforme (NVENC / QSV / AMF / VideoToolbox), optimisés pour la réactivité plutôt que pour le confort.
- **Architecture découplée en 46 sous-modules** — des composants clairement séparés, avec une approche native des conteneurs : chaque service, base de données, build, test et analyse s'exécute dans un conteneur.

Principaux défis techniques et solutions apportées

- **Streaming à faible latence sur du matériel hétérogène.** Chaque OS et GPU expose la capture et l'encodage différemment, et la latence ne pardonne pas. Solution : un chemin de capture/encodage conscient de la plateforme — DXGI / ScreenCaptureKit / PipeWire alimentant NVENC / QSV / AMF / VideoToolbox — transporté via WebRTC / QUIC / UDP, de sorte que chaque machine utilise sa propre voie native la plus rapide pour accéder aux pixels.
- **Un seul produit pour desktop, mobile, TV et web.** Solution : des clients triples (Wails, Flutter, Angular) partageant un seul cœur Go compilé en WASM pour le navigateur, ce qui permet à une correction ou une fonctionnalité écrite une fois d'apparaître sur les quatre interfaces au lieu d'être portée quatre fois.
- **Fonctionnement multi-locataire en marque blanche.** Solution : intégration directe dans le backend central de la personnalisation par locataire, du filtrage du catalogue,

d'OAuth2 et de la facturation, de sorte que l'isolation des locataires et la personnalisation de la marque soient des primitives de la plateforme plutôt que des forks par client.

Pile technologique

- **Go (version racine 1.26.2 / sous-modules 1.25+)** — le cœur partagé du backend et de l'agent hôte ; un seul langage qui se compile en binaires natifs *et* en WASM, ce qui rend possible la conception mono-cœur et multi-clients.
- **Wails v2** — le client desktop, liant le cœur Go à une webview intégrée afin que l'application desktop réutilise directement la logique du cœur au lieu de la réimplémenter.
- **Flutter 3.29+** — le client mobile/TV, appelant le cœur Go via FFI pour une interface native sur smartphones et téléviseurs, sans nécessiter un second backend.
- **Angular 17+** — le client web, exécutant le même cœur Go compilé en WASM, ce qui fait du navigateur une surface de premier plan, et non une version allégée.
- **WebRTC / Pion v4, QUIC / quic-go, UDP personnalisé** — trois protocoles de transport en temps réel permettant à la plateforme de choisir le chemin à la latence la plus faible pour chaque réseau et client.
- **Encodeurs matériels (NVENC / QSV / AMF / VideoToolbox) et capture par plateforme (DXGI / ScreenCaptureKit / PipeWire)** — le chemin de capture et d'encodage accéléré par GPU, sélectionné par plateforme afin que l'encodage ne soit jamais limité par le CPU.
- **Conteneurs (Docker/Podman)** — chaque service, base de données, build, test et analyse s'exécute dans un conteneur, rendant l'ensemble du système reproductible en termes de déploiement et de vérification.
- **mDNS / rendez-vous** — découverte automatique des hôtes sans configuration, permettant aux clients de localiser automatiquement leur hôte de streaming sur le réseau.

Contenu

Notes sur le statut et l'honnêteté

- **Statut : bêta.** Les objectifs de latence du README (≤ 30 ms en LAN / ≤ 50 ms en WAN pour le p999), le cadrage « niveau console / niveau PS4 Pro » et le nombre de cellules de la matrice de tests sont les cibles de conception déclarées par le projet lui-même, non vérifiées de manière indépendante, et présentées comme telles.

- **Licence : à déterminer.** Aucune LICENCE n'a été détectée via les références GitHub et API — NON VÉRIFIÉ / non déclarée.

Niveau de priorité : Helix-primaire.